

Language Models

CENG 493

Language Models

- A **language model** is a statistical or **machine learning** model trained to conduct a probability distribution over words.
- The primary purpose of a language model is to capture the statistical properties of natural language.
- By learning the probability distribution of word sequences, a language model can predict the likelihood of a given word following a sequence of words.
- Language models can be broadly categorized into two types:
 - Pure Statistical Methods
 - Neural Models

Language Models

- Pure Statistical Methods
 - N-gram models
 - Exponential Models (i.e. Maximum Entropy model)
 - Skip-gram models
- Neural Models
 - Recurrent Neural Networks
 - LSTM
 - GRU
 - Transformer based models
 - BERT
 - Large Language Models (LLMs)

Language Models

What language models can do

- Content generation
- Part-of-speech (POS) tagging
- Question answering
- Text summarization
- Sentiment analysis
- Machine translation
- Code completion
- Conversational AI

What language models can't do

- They can't perform tasks that involve
 - common-sense knowledge,
 - understanding abstract concepts, and
 - making inferences based on incomplete information.
- They also lack the ability to
 - understand the world as humans do, and they can't make decisions or take actions in the physical world.

Language Model

A model for how humans generate language

- Places a probability distribution over any sequence of words
- By construction, it also provides a model for generating text according to its distribution

Used in many language-oriented tasks, e.g.,

- Machine translation: $P(\text{high winds tonight}) > P(\text{large winds tonight})$
- Spelling correction: $P(\text{about 15 minutes}) > P(\text{about 15 minuets})$
- Speech recognition: $P(\text{I saw a van}) \gg P(\text{eyes awe of an})$

Language Model

- Probability distributions over sentences (i.e., word sequences)

$$P(W) = P(w_1 w_2 w_3 w_4 \dots w_k)$$

- Can use them to **generate** strings

$$P(w_k \mid w_2 w_3 w_4 \dots w_{k-1})$$

- **Rank** possible sentences

- $P(\text{"Today is Tuesday"}) > P(\text{"Tuesday Today is"})$

- $P(\text{"Today is Tuesday"}) > P(\text{"Today is Atlanta"})$

Statistical Language Processing

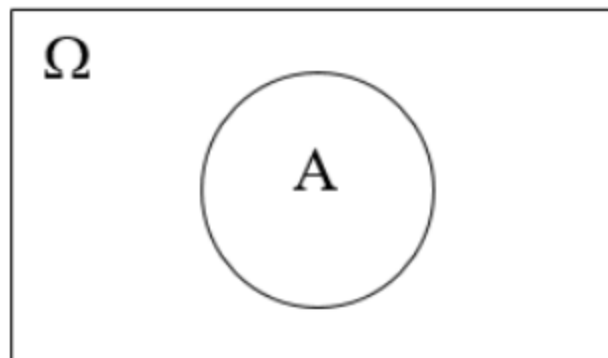
- In the solution of many problems in the natural language processing, **statistical language processing** techniques can be also used.
 - optical character recognition
 - spelling correction
 - speech recognition
 - machine translation
 - part of speech tagging
 - parsing
- Statistical techniques can be used to disambiguate the input.
- They can be used to select the most probable solution.
- Statistical techniques depend on the probability theory.
- To able to use statistical techniques, we will need corpora to collect statistics.
- Corpora should be big enough to capture the required knowledge.

Basic Probability

- **Probability Theory:** predicting how likely it is that something will happen.
- **Probabilities:** numbers between 0 and 1.
- **Probability Function:**
 - $P(A)$ means that how likely the event A happens.
 - $P(A)$ is a number between 0 and 1
 - $P(A)=1 \Rightarrow$ a certain event
 - $P(A)=0 \Rightarrow$ an impossible event
- **Example:** a coin is tossed three times. What is the probability of 3 heads?
 - $1/8$
 - uniform distribution

Probability Spaces

- There is a sample space and the subsets of this sample space describe the events.
- Ω is a sample space.
 - Ω is the certain event
 - the empty set is the impossible event.



$P(A)$ is between 0 and 1

$$P(\Omega) = 1$$

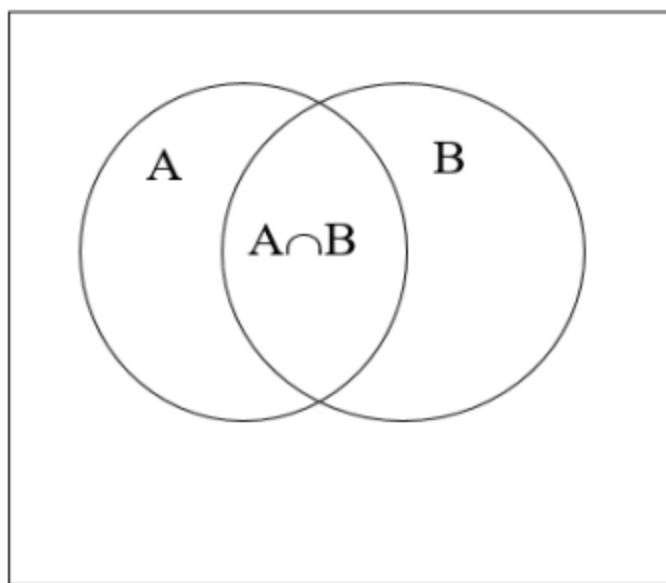
Unconditional and Conditional Probability

- **Unconditional Probability or Prior Probability**
 - $P(A)$
 - the probability of the event A does not depend on other events.
- **Conditional Probability -- Posterior Probability -- Likelihood**
 - $P(A|B)$
 - this is read as the probability of A given that we know B .

Example:

- $P(\text{put})$ is the probability of to see the word *put* in a text
 - $P(\text{on}|\text{put})$ is the probability of to see the word *on* after seeing the word *put*.
-
- **Joint Probability**
 - $P(A \cap B)$ or $P(A, B)$
 - the probability of the events A and B occur together

Unconditional and Conditional Probability



$$P(A|B) = P(A \cap B) / P(B)$$

$$P(B|A) = P(A \cap B) / P(A)$$

Bayes' Theorem

- Bayes' theorem is used to calculate $P(A|B)$ from given $P(B|A)$.
- We know that:

$$P(A \cap B) = P(A|B) P(B)$$

$$P(A \cap B) = P(B|A) P(A)$$

- So, we will have:

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

$$P(B|A) = \frac{P(A|B) P(B)}{P(A)}$$

Bayes' Theorem

- Bayes' Theorem is a way of finding a probability when we know certain other probabilities.
- The formula is:

$$P(A|B) = \frac{P(A) P(B|A)}{P(B)}$$

Which tells us: how often A happens *given that B happens*, written **P(A|B)**,
When we know: how often B happens *given that A happens*, written **P(B|A)**
and how likely A is on its own, written **P(A)**
and how likely B is on its own, written **P(B)**

Bayes' Theorem

Imagine 100 people at a party, and you tally how many wear pink or not, and if a man or not, and get these numbers:

	Pink	notPink
Man	5	35
notMan	20	40

Let us do some totals:

	Pink	notPink	
Man	5	35	40
notMan	20	40	60
	25	75	100

And calculate some probabilities:

- the probability of being a man is $P(\text{Man}) = \frac{40}{100} = 0,4$
- the probability of wearing pink is $P(\text{Pink}) = \frac{25}{100} = 0,25$
- the probability that a man wears pink is $P(\text{Pink}|\text{Man}) = \frac{5}{40} = 0,125$
- the probability that a person wearing pink is a man **$P(\text{Man}|\text{Pink}) = \dots$**

- **$P(\text{Man}) = 0,4$,**
- **$P(\text{Pink}) = 0,25$ and**
- **$P(\text{Pink}|\text{Man}) = 0,125$**

Can you discover **$P(\text{Man}|\text{Pink})$** ?

$$P(\text{Man}|\text{Pink}) = \frac{P(\text{Man}) P(\text{Pink}|\text{Man})}{P(\text{Pink})}$$

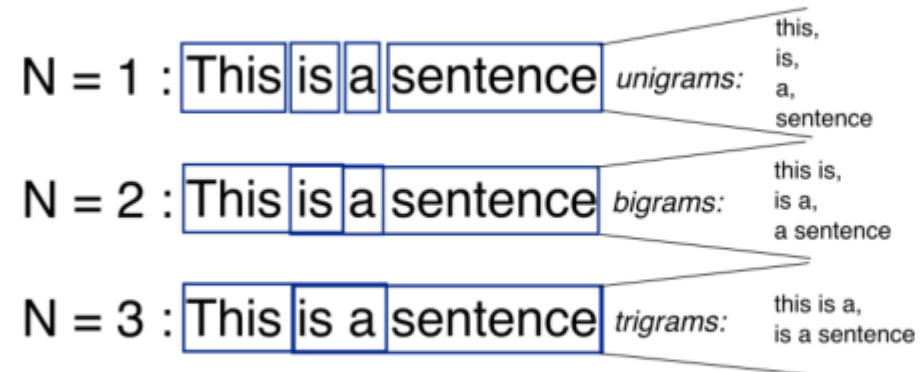
$$P(\text{Man}|\text{Pink}) = \frac{0,4 \times 0,125}{0,25} = 0,2$$

Back to Language Models (LMs)

- Models that assign probabilities to sequences of words are called **language models (LMs)**.
- The simplest language model that assigns probabilities to sentences and sequences of words is **n-gram language model**.
- An **n-gram** is a sequence of N words:
 - A 1-gram (unigram) is a single word sequence of words like “please” or “turn”.
 - A 2-gram (bigram) is a two-word sequence of words like “please turn”, “turn your”, or “your homework”.
 - A 3-gram (trigram) is a three-word sequence of words like “please turn your”, or “turn your homework”.
- We can use n-gram models to estimate the probability of the last word of an n-gram given the previous words, and also to assign probabilities to entire word sequences.

N-grams

N-grams: a contiguous sequence of n tokens from a given piece of text



<http://recognize-speech.com/language-model/n-gram-model/comparison>

Probabilistic Language Models

- Our goal is to compute the probability of a sentence or sequence of words $W (=w_1, w_2, \dots, w_n)$:

$$P(W) = P(w_1, w_2, \dots, w_n)$$

- What is the probability of an upcoming word?:
 - $P(w_5 | w_1, w_2, w_3, w_4)$
- A model that computes either of these:
 - $P(W)$ or $P(w_n | w_1, w_2 \dots w_{n-1})$ is called a **language model**.

How to compute $P(W)$

- How to compute this joint probability:
 - $P(\text{its, water, is, so, transparent, that})$
- Intuition: let's rely on the Chain Rule of Probability

Chain Rule of Probability

- *How can we compute probabilities of entire word sequences like $w_1, w_2, \dots, w_n$?*
 - The probability of the word sequence $w_1, w_2, \dots, w_n$ is $P(w_1, w_2, \dots, w_n)$.
- We can use the **chain rule of the probability** to decompose this probability:

$$\begin{aligned} P(w_1^n) &= P(w_1) P(w_2|w_1) P(w_3|w_1^2) \dots P(w_n|w_1^{n-1}) \\ &= \prod_{k=1}^n P(w_k | w_1^{k-1}) \end{aligned}$$

Example:

$P(\text{the man from jupiter}) =$

$P(\text{the}) P(\text{man}|\text{the}) P(\text{from}|\text{the man}) P(\text{jupiter}|\text{the man from})$

Chain Rule of Probability and Conditional Probabilities

- The chain rule shows the link between computing the joint probability of a sequence and computing the conditional probability of a word given previous words.
- Definition of Conditional Probabilities:

$$P(B|A) = P(A,B) / P(A) \quad \rightarrow \quad P(A,B) = P(A) P(B|A)$$

- Conditional Probabilities with More Variables:

$$P(A,B,C,D) = P(A) P(B|A) P(C|A,B) P(D|A,B,C)$$

- **Chain Rule:**

$$P(w_1 \dots w_n) = P(w_1) P(w_2|w_1) P(w_3|w_1 w_2) \dots P(w_n|w_1 \dots w_{n-1})$$

Computing Conditional Probabilities

- *To compute the exact probability of a word given a long sequence of preceding words is difficult (sometimes impossible).*
- We are trying to compute $P(w_n | w_1 \dots w_{n-1})$ which is the **probability of seeing w_n after seeing w_1^{n-1}** .
- We may try to compute $P(w_n | w_1 \dots w_{n-1})$ **exactly** as follows:

$$P(w_n | w_1 \dots w_{n-1}) = \text{count}(w_1 \dots w_{n-1} w_n) / \text{count}(w_1 \dots w_{n-1}) \quad \Rightarrow \quad P(\text{the} \mid \text{its water is so transparent that}) = \frac{\text{Count}(\text{its water is so transparent that the})}{\text{Count}(\text{its water is so transparent that})}$$

- Too many possible sentences and we may never see enough data for estimating these probability values.
- So, we need to compute $P(w_n | w_1 \dots w_{n-1})$ **approximately**.

Markov Assumption

The assumption that the probability of a word depends only on the previous word(s) is called **Markov assumption**

Simplifying assumption:

(the | its water is so transparent that) $\approx$ P(the | that)



[Andrey Markov](#)

Or maybe

P(the | its water is so transparent that) $\approx$ P(the | transparent that)

$$P(w_1 w_2 \dots w_n) \approx \prod_i P(w_i | w_{i-k} \dots w_{i-1})$$

In other words, we approximate each component in the product

$$P(w_i | w_1 w_2 \dots w_{i-1}) \approx P(w_i | w_{i-k} \dots w_{i-1})$$

Simplest Case- Unigrams

$$P(w_1 w_2 \dots w_n) \approx \prod_i P(w_i)$$

Some automatically generated sentences from a unigram model

fifth, an, of, futures, the, an, incorporated, a, a, the, inflation, most,
dollars, quarter, in, is, mass

thrift, did, eighty, said, hard, 'm, july, bullish

that, or, limited, the

Bigram (2-gram)

Condition on the previous word:

$$P(w_i | w_1 w_2 \dots w_{i-1}) \approx P(w_i | w_{i-1})$$

texaco, rose, one, in, this, issue, is, pursuing, growth, in, a, boiler, house, said, mr.,
gurria, mexico, 's, motion, control, proposal, without, permission, from, five, hundred,
fifty, five, yen

outside, new, car, parking, lot, of, the, agreement, reached

this, would, be, a, record, november

N-grams

We can extend to trigrams, 4-grams, 5-grams

In general, this is an insufficient model of language because language has long-distance dependencies:

- “The computer which I had just put into the machine room on the fifth floor crashed.”

But we can often get away with N-gram models

N-Grams

- The intuition of the n-gram model (simplifying assumption):
 - instead of computing the probability of a word given its entire history, we can approximate the history by just the last few words.

$$P(w_n | w_1 \dots w_{n-1}) \approx P(w_n) \quad \text{unigram}$$

$$P(w_n | w_1 \dots w_{n-1}) \approx P(w_n | w_{n-1}) \quad \text{bigram}$$

$$P(w_n | w_1 \dots w_{n-1}) \approx P(w_n | w_{n-1} w_{n-2}) \quad \text{trigram}$$

$$P(w_n | w_1 \dots w_{n-1}) \approx P(w_n | w_{n-1} w_{n-2} w_{n-3}) \quad \text{4-gram}$$

$$P(w_n | w_1 \dots w_{n-1}) \approx P(w_n | w_{n-1} w_{n-2} w_{n-3} w_{n-4}) \quad \text{5-gram}$$

- In general, **N-Gram** is

$$P(w_n | w_1 \dots w_{n-1}) \approx P(w_n | w_{n-N+1}^{n-1})$$

N-Grams

computing probabilities of word sequences

Unigrams --
$$P(w_1^n) \approx \prod_{k=1}^n P(w_k)$$

Bigrams --
$$P(w_1^n) \approx \prod_{k=1}^n P(w_k \mid w_{k-1})$$

Trigrams --
$$P(w_1^n) \approx \prod_{k=1}^n P(w_k \mid w_{k-1}w_{k-2})$$

4-grams --
$$P(w_1^n) \approx \prod_{k=1}^n P(w_k \mid w_{k-1}w_{k-2}w_{k-3})$$

N-Grams

computing probabilities of word sequences (Sentences)

Unigram

$$P(< s> \text{ the man from jupiter came } </ s>) \approx \\ P(\text{the}) P(\text{man}) P(\text{from}) P(\text{jupiter}) P(\text{came})$$

Bigram

$$P(< s> \text{ the man from jupiter came } </ s>) \approx \\ P(\text{the} | < s>) P(\text{man} | \text{the}) P(\text{from} | \text{man}) P(\text{jupiter} | \text{from}) P(\text{came} | \text{jupiter}) P(</ s> | \text{came})$$

Trigram

$$P(< s> \text{ the man from jupiter came } </ s>) \approx \\ P(\text{the} | < s> \text{ } < s>) P(\text{man} | < s> \text{ the}) P(\text{from} | \text{the man}) P(\text{jupiter} | \text{man from}) \\ P(\text{came} | \text{from jupiter}) P(</ s> | \text{jupiter came}) P(</ s> | \text{came } </ s>)$$

N-Grams and Markov Models

- The assumption that the probability of a word depends only on the previous word(s) is called **Markov assumption**.
- **Markov models** are the class of probabilistic models that assume that we can predict the probability of some future unit without looking too far into the past.
- A **bigram** is called a **first-order** Markov model (because it looks one token into the past);
- A **trigram** is called a **second-order** Markov model;
- In general a **N-Gram** is called a **N-1 order** Markov model.

Estimating N-Gram Probabilities

- Estimating n-gram probabilities is called **maximum likelihood estimation** (or **MLE**).
- We get the MLE estimate for the parameters of an n-gram model by *getting counts from a corpus*, and **normalizing** the counts so that they lie between 0 and 1.
- Estimating bigram probabilities:

$$P(w_n | w_{n-1}) = \frac{C(w_{n-1}w_n)}{\sum_w C(w_{n-1}w)} = \frac{C(w_{n-1}w_n)}{C(w_{n-1})} \quad \text{where } C \text{ is the count of that pattern in the corpus}$$

- Estimating N-Gram probabilities

$$P(w_n | w_{n-N+1}^{n-1}) = \frac{C(w_{n-N+1}^{n-1}w_n)}{C(w_{n-N+1}^{n-1})}$$

Estimating N-Gram Probabilities

A Bigram Example

- ***A mini-corpus:*** We augment each sentence with a special symbol $\langle s \rangle$ at the beginning of the sentence, to give us the bigram context of the first word, and special end-symbol $\langle /s \rangle$.
 $\langle s \rangle$ I am Sam $\langle /s \rangle$
 $\langle s \rangle$ Sam I am $\langle /s \rangle$
 $\langle s \rangle$ I fly $\langle /s \rangle$
- ***Unique words:*** I, am, Sam, fly
- ***Bigrams:*** $\langle s \rangle$ and $\langle /s \rangle$ are also tokens. There are $6(4+2)$ tokens and $6*6=36$ bigrams

$P(I \langle s \rangle)=2/3$	$P(\text{Sam} \langle s \rangle)=1/3$	$P(\text{am} \langle s \rangle)=0$	$P(\text{fly} \langle s \rangle)=0$	$P(\langle s \rangle \langle s \rangle)=0$	$P(\langle /s \rangle \langle s \rangle)=0$
$P(I I)=0$	$P(\text{Sam} I)=0$	$P(\text{am} I)=2/3$	$P(\text{fly} I)=1/3$	$P(\langle s \rangle I)=0$	$P(\langle /s \rangle I)=0$
$P(I \text{am})=0$	$P(\text{Sam} \text{am})=1/2$	$P(\text{am} \text{am})=0$	$P(\text{fly} \text{am})=0$	$P(\langle s \rangle \text{am})=0$	$P(\langle /s \rangle \text{am})=1/2$
$P(I \text{Sam})=1/2$	$P(\text{Sam} \text{Sam})=0$	$P(\text{am} \text{Sam})=0$	$P(\text{fly} \text{Sam})=0$	$P(\langle s \rangle \text{Sam})=0$	$P(\langle /s \rangle \text{Sam})=1/2$
$P(I \text{fly})=0$	$P(\text{Sam} \text{fly})=0$	$P(\text{am} \text{fly})=0$	$P(\text{fly} \text{fly})=0$	$P(\langle s \rangle \text{fly})=0$	$P(\langle /s \rangle \text{fly})=1$
$P(I \langle /s \rangle)=0$	$P(\text{Sam} \langle /s \rangle)=0$	$P(\text{am} \langle /s \rangle)=0$	$P(\text{fly} \langle /s \rangle)=0$	$P(\langle s \rangle \langle /s \rangle)=1 \text{ or } ?$	$P(\langle /s \rangle \langle /s \rangle)=0$

Estimating N-Gram Probabilities

Example

<s> I am Sam </s>

<s> Sam I am </s>

<s> I fly </s>

- **Unigrams:** I, am, Sam, fly

$$P(I)=3/8$$

$$P(am)=2/8$$

$$P(Sam)=2/8$$

$$P(fly)=1/8$$

- **Trigrams:** There are $6*6*6=216$ trigrams.

– Assume there are two tokens <s> <s> at the begining, and two tokens </s> </s> at the end.

$$P(I|<s> <s>)=2/3$$

$$P(Sam|<s> <s>)=1/3$$

$$P(am|<s> I)=1/2$$

$$P(fly|<s> I)=1/2$$

$$P(I|<s> Sam)=1$$

$$P(Sam|I am)=1/2$$

$$P(</s>|I am)=1/2$$

$$P(</s>|am Sam)=1$$

$$P(</s>|Sam </s>)=1$$

Estimating N-Gram Probabilities

Example

<s> I am Sam </s>

<s> Sam I am </s>

<s> I fly </s>

- **Unigrams:** I, am, Sam, fly

$$P(I)=3/8$$

$$P(am)=2/8$$

$$P(Sam)=2/8$$

$$P(fly)=1/8$$

- **Trigrams:** There are $6*6*6=216$ trigrams.
 - Assume there are two tokens <s> <s> at the begining, and two tokens </s> </s> at the end.

$$P(I|<s> <s>)=2/3$$

$$P(Sam|<s> <s>)=1/3$$

$$P(am|<s> I)=1/2$$

$$P(fly|<s> I)=1/2$$

$$P(I|<s> Sam)=1$$

$$P(Sam|I am)=1/2$$

$$P(</s>|I am)=1/2$$

$$P(</s>|am Sam)=1$$

$$P(</s>|Sam </s>)=1$$

Estimating N-Gram Probabilities - Example 2: Berkeley Restaurant Project sentences

A dialogue system from the last century that answered questions about a database of restaurants in Berkeley, California

Here are some text-normalized sample user queries

- can you tell me about any good cantonese restaurants close by
- mid priced thai food is what i'm looking for
- tell me about chez panisse
- can you give me a listing of the kinds of food that are available
- i'm looking for a good place to eat breakfast
- when is caffe venezia open during the day

Estimating N-Gram Probabilities - Example 2: Berkeley Restaurant Project sentences

- There are 9332 sentences in the corpus.
- Raw biagram counts of 8 words (out of 1446 words)

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Estimating N-Gram Probabilities - Example 2: Berkeley Restaurant Project sentences

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

- Unigram counts:

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

- Normalize bigrams by unigram counts:

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Bigram Estimates of Sentence Probabilities

- Some other bigrams:

$$P(i|<s>)=0.25$$

$$P(\text{english}|\text{want})=0.0011$$

$$P(\text{food}|\text{english})=0.5$$

$$P(</s>|\text{food})=0.68$$

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

- Compute the probability of sentence **I want English food**

$$P(<s> i \text{ want english food } </s>)$$

$$= P(i|<s>) P(\text{want}|i) P(\text{english}|\text{want}) P(\text{food}|\text{english}) P(</s>|\text{food})$$

$$= 0.25 * 0.33 * 0.0011 * 0.5 * 0.68$$

$$= 0.000031$$

Log Probabilities

- Since probabilities are less than or equal to 1, the more probabilities we multiply together, the *smaller the product becomes*.
 - Multiplying enough n-grams together would result in **numerical underflow**.
- By using **log probabilities** instead of *raw probabilities*, we get numbers that are not as small.
 - Adding in log space is equivalent to multiplying in linear space, so we combine log probabilities by adding them.
 - adding is faster than multiplying
 - The result of doing all computation and storage in log space is that we only need to convert back into probabilities if we need to report them at the end

$$\log(p_1 \times p_2 \times p_3 \times p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

$$p_1 \times p_2 \times p_3 \times p_4 = \exp (\log p_1 + \log p_2 + \log p_3 + \log p_4)$$

Evaluating Language Models

- Does our language model prefer good sentences to bad ones?
 - Assign higher probability to “real” or “frequently observed” sentences than “ungrammatical” or “rarely observed” sentences?
- We train parameters of our model on a **training set**.
- We test the model’s performance on data we haven’t seen.
 - A **test set** is an unseen dataset that is different from our training set, totally unused.
- An **evaluation metric** tells us how well our model does on the *test set*.

Evaluating Language Models

Extrinsic Evaluation

- **Extrinsic Evaluation** of a N-gram *language model* is to use it in an application and measure how much the application improves.
- To compare two language models A and B:
 - Use each of language model in a task such as spelling corrector, MT system.
 - Get an accuracy for A and for B
 - How many misspelled words corrected properly
 - How many words translated correctly
 - Compare accuracy for A and B
 - The model produces the better accuracy is the better model.
- Extrinsic evaluation can be time-consuming.

Evaluating Language Models

Intrinsic Evaluation

- An **intrinsic evaluation** metric is one that measures the quality of a model independent of any application.
- When a corpus of text is given and to compare two different n-gram models,
 - Divide the data into *training* and *test sets*,
 - Train the parameters of both models on the *training set*, and
 - Compare how well the two trained models fit the test set.
 - Whichever model assigns a higher probability to the test set
- In practice, *probability-based metric* called **perplexity** is used instead of raw probability as our metric for evaluating language models.

Evaluating Language Models

Perplexity

- The *best language model* is one that best predicts an unseen test set
 - Gives the highest $P(\text{testset})$
- The **perplexity** of a language model on a test set is the *inverse probability of the test set*, normalized by the number of words.
- **Minimizing perplexity is the same as maximizing probability**
- The **perplexity PP** for a test set $W=w_1w_2\dots w_N$ is

$$PP(W) = \sqrt[N]{\frac{1}{P(w_1w_2\dots w_N)}} = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i|w_1\dots w_{i-1})}} \quad \text{by chain rule}$$

- The **perplexity PP** for bigrams:

$$PP(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i|w_{i-1})}}$$

Evaluating Language Models

Perplexity

- Lower perplexity = better model
- Training 38 million words, test 1.5 million words, WSJ

	Unigram	Bigram	Trigram
Perplexity	962	170	109

- An intrinsic improvement in perplexity does not guarantee an (extrinsic) improvement in the performance of a language processing task like speech recognition or machine translation.
 - Nonetheless, because perplexity often correlates with such improvements, it is commonly used as a quick check on an algorithm.
 - But a model's improvement in perplexity should always be confirmed by an end-to-end evaluation of a real task before concluding the evaluation of the model.

Sampling Sentences from a Language Model

- **Sampling** from a distribution means to choose random points according to their likelihood.
- **Sampling from a language model** means to generate some sentences, choosing each sentence according to its likelihood as defined by the model.
- More likely to generate sentences that the model thinks have a high probability and less likely to generate sentences that the model thinks have a low probability.

Sampling Sentences from a Language Model

- a bigram model example

- Choose a random bigram ($\langle s \rangle$, w) according to its probability $p(w|\langle s \rangle)$
 $\langle s \rangle$ I
- Now choose a random bigram (I , x) according to its probability $p(x|I)$
 $\langle s \rangle$ I want
- Now choose a random bigram (want , x) according to its probability $p(x|\text{want})$
 $\langle s \rangle$ I want to
- Now choose a random bigram (to , x) according to its probability $p(x|\text{to})$
 $\langle s \rangle$ I want to eat
- Now choose a random bigram (eat , x) according to its probability $p(x|\text{eat})$
 $\langle s \rangle$ I want to eat Chinese
- Now choose a random bigram (Chinese , x) according to its probability $p(x|\text{Chinese})$
 $\langle s \rangle$ I want to eat Chinese food
- Now choose a random bigram (food , x) according to its probability $p(x|\text{food})$
 $\langle s \rangle$ I want to eat Chinese food $\langle /s \rangle$

Generated Sentence: $\langle s \rangle$ I want to eat Chinese food $\langle /s \rangle$

Unknown Words

- We have to deal with words we haven't seen before, which we call **unknown words**.
- We can model these potential unknown words in the test set by adding a **pseudo-word called <UNK>** into our training set too.
- One way to handle unknown words is:
 - Replace words in the training data by <UNK> based on their frequency.
 - For example we can replace by <UNK> all words that occur fewer than n times in the training set, where n is some small number, or
 - Equivalently select a vocabulary size V in advance (say 50,000) and choose the top V words by frequency and replace the rest by <UNK>.
 - Proceed to train the language model as before, treating <UNK> like a regular word.

Generalization and Zeros

- The **n-gram model**, like many statistical models, is *dependent on the training corpus*.
 - the probabilities often encode specific facts about a given training corpus.
- N-grams only work well for word prediction if the *test corpus looks like the training corpus*
 - In real life, it often doesn't
 - We need to train robust models that **generalize**!
 - One kind of generalization: Getting rid of **Zeros**!
 - Things that don't ever occur in the training set, but occur in the test set
- **Zeros**: things that don't ever occur in the training set but do occur in the test set causes problem for two reasons.
 - First, underestimating the probability of all sorts of words that might occur,
 - Second, if probability of any word in test set is 0, entire probability of test set is 0.

Smoothing

- To keep a language model from assigning zero probability to these unseen events, we'll have to *shave off a bit of probability mass from some more frequent events and give it to the events we've never seen*.
- This modification is called **smoothing** (or **discounting**).
- There are many ways to do **smoothing**, and some of them are:
 - **Add-1 smoothing (Laplace Smoothing)**
 - **Add-k smoothing,**
 - **Backoff**
 - **Kneser-Ney smoothing.**

Laplace Smoothing (Add-1 Smoothing)

Laplace Smoothing for Unigrams:

- The unsmoothed maximum likelihood estimate of the unigram probability of the word w_i is its count c_i normalized by the total number of word tokens N :

$$P(w_i) = c_i / N$$

- **Laplace smoothing** adds one to each count. Since there are V words in the vocabulary and each one was incremented, we also need to adjust the denominator to take into account the extra V observations.

$$P_{\text{Laplace}}(w_i) = (c_i + 1) / (N + V)$$

References

- NLP Course slides from
 - University of Southern Maine
(<https://cs.usm.maine.edu/~behrooz.mansouri/>)
 - Hacettepe University
(<https://web.cs.hacettepe.edu.tr/~ilyas/Courses/BIL711/>)
 - Stanford University
(<https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1234/>)